Units Speed Matters ! An End-to-End Case Study

(1) Key Assumption : Local Linear Approximation

(1) What does it mean in A/B Testing ?

In the context of experimentation (A/B testing), Local Linear Approximation means:

> When evaluating a small performance change (like 100ms slowdown on a webpage), we assume that the impact on key metrics (like user engagement, conversion rate, revenue) changes linearly in that small neighborhood.

It's like saying:

> "If a 100ms slowdown reduces conversion by 1%, then a 200ms slowdown might reduce it by roughly 2%."

(2) Mathematical Insights

→ This assumes that in a small local range of values (small changes in page speed), the relationship between input (delay) and output (metric) behaves approximately like a straight line

This is essentially a first-order Taylor approximation

$$f(x + \Delta x) \approx f(x) + f'(x) \cdot \Delta x$$

In experiments,

- x : original page load time

- Δx = Change (e.g., +100ms)
- $f(x)$ = metric like revenue, conversion.
- $f'(x)$ = slope, i.e. how sensitive metric is to change.

Example:

Let's say the real (but unknown to us) conversion rate function is:

$$f(x) = 4.0 - 0.001x + 0.000002x^2$$

where:

- $f(x)$ = conversion rate (%)
- x = page load delay in milliseconds.

We know:

- $f(100) = ?$
- Estimate $f(120)$ using linear approximation around 100

Step 1: Compute $f(100)$.

$$f(100) = 4.0 - 0.001(100) + 0.000002(100)^2$$
$$= 4.0 - 0.1 + 0.000002(10000)$$
$$= 4.0 - 0.1 + 0.02$$
$$= 3.92$$

So, conversion rate at 100ms delay is 3.92%

Step 2: Compute the Derivative $f'(x)$

$$f(x) = 4.0 - 0.001x + 0.000002x^2$$
$$f'(x) = -0.001 + 0.000004x$$

Step3 : Plug in $x = 100$ into $f'(x)$

$$f'(100) = -0.001 + 0.000004(100)$$
$$= -0.001 + 0.0004 = -0.0006.$$

This is the slope of the curve at 100ms:
For every 1ms added, the conversion drops by 0.0006%.

Step4: Linear Approximation formula.

Use:

$$f(x + \Delta x) \approx f(x) + f'(x) \cdot \Delta x$$

We want to estimate $f(120)$, so $\Delta x = 20$.

$$f(120) \approx f(100) + f'(100) \cdot 20.$$
$$= 3.92 + (-0.0006) \cdot 20$$
$$= 3.92 - 0.012 = \boxed{3.908}$$

Step5: Compare with True Values
Now, let's compute the true value at 120ms to compare.

$$f(120) = 4.0 - 0.001(120) + 0.000002(120)^2.$$
$$= 3.9088$$

Conclusion:

Method	Result.
Linear Approx	3.908
True value	3.9088

⇒ Linear approximation is very close when the step is small (20ms)

(3) Real-world Ancology.

Scenario:

imagine you're testing how users respond to sound volume in an app

- You slowly increase the volume from 50 to 60.
- You notice: users drop off slightly - maybe because it's too loud.
- You see a small, predictable change for each unit increase, 0.5% more users leave.

You now assume:

"If I increase the volume just a little more (like from 60 to 62), users will react in the same way - about 0.5% more will leave per unit."

That's Local Linear Approximation

- For small changes, assume the effect is constant per unit (like a straight line)
- You don't need to test every volume level - you trust the local slope.

In A/B Testing Terms:

Let's replace volume with page load delay:

- You increase delay from 100ms → 110ms.
- Conversion rate drops from 3.92% → 3.86% → That's a 0.06% drop over 10ms.

Now you ask:

> "what will happen at 120 ms?"

You assume the effects continue linearly, so you estimate:

$$\text{Drop per ms} = \frac{3.92 - 3.86}{10} = 0.006\% \text{ per ms}$$

$$\text{Estimated at 120ms} = 3.86 - (0.006 \times 10) = 3.80\%$$

This works because:

- The conversion curve is smooth locally.
- You're not making a big leap — just a small extrapolation.

Key idea:

> If the curve of user behaviour doesn't bend sharply in the small zone you're testing, it's safe to approximate it as a straight line.

That's the core idea of local linear approximation in experimentation.

(4) Related Concept

This concept is highly linked to:

- Sensitivity Analysis — Estimating how much a metric changes when we tweak input.
- Gradient Descent / Derivative in ML — Local slope used to optimize
- Causal Inference — We're modeling effect of treatment assuming a local smooth behavior.

(5) Overlapping Concept from Later Section:

> From "Extreme Results" (same unit), they later warn that: small changes may show small effect (linear), but larger slowdowns can impact differently, e.g. bounce rates skyrocket — no longer linear.

So both sections interlock:

- Use linear approx locally, but understand its limitations when effects are large.

(2) How to Measure Website Performance

(1) Why it matters?

→ When you're testing how speed affect users (e.g by intentionally slowing down page), you need to measure performance precisely and meaningfully. Not just how slow it is — but how the user experience that slowness.

(2) What is "Website Performance"?

→ It refers to how fast and smoothly a web page loads and becomes usable for the user.

There are multiple layers of performance:

Layer	Example	Meaning
Server-side time	Backend took 200ms to respond	How fast the server responds.

Page load time	2.3 seconds	Time from click to full page.
Above-the-fold load	Hero image/text loaded in 1.2s	What user sees first
First Contentful Paint (FCP)	Text appears in 0.8s	First meaning thing visible
Time to Interactive (TTI)	3.0 seconds	Page is fully usable.

In the book, they stress that user-perceived performance matters more than technical-only metrics.

Core idea from the book:
Different pages and users have different load times so we need to aggregate performance carefully — don't just average anything blindly.

(3) Key Concept: Percentiles > Averages.

Why is this important?
When you're running experiments (like slowing down a site to study impact), you want to summarize user performance experience.
You might think:
"Let's just average all users' page load times and compare"
But that's dangerous and misleading — here's why:

⚠ Averages lie in Skewed Distributions

Example

Suppose 10 users have these page load times (in seconds):

[1.8, 2.0, 2.1, 2.2, 2.3, 2.4, 2.5, 9.0, 10.0]

- Mean (average) = $\frac{1.8 + 2.0 + \cdots + 10.0}{10} \approx 3.66$ seconds
- Median (50th percentile) = 2.3 seconds

Problem:

- The mean is pulled upward by just 2 outlier users (9.0s and 10.0s)
- Most users experience a page load of ~2.3 seconds, not 3.7!

That's why percentiles give a more honest view of user experience.

④ So, how to measure performance?

☑ Use metrics like:

Metric Name	Description
page.loadTime	Total time from click to full page render
navTiming.domComplete	When page is fully parsed.
First Paint, First Content ful Paint	Time to show something meaningful
TTFB (Time to First Byte)	Server response time

TTI (Time to Interactive) Page usability.

⚠ Important.

Don't use only technical telemetry. You need:

- Real user data
- Tracked across variations (e.g. A vs B groups)
- Captured at scale

So you're not just measuring "how fast", but "how fast it feels to users".

(5) Related Concept Overlap.

This ties into:

- Local Linear Approximation: Once you measure the slowdown, you assume small changes in load time have small, proportional impacts.
- Slowdown Experiment Design: You must have reliable performance data to design and analyze your treatment group.

(6) Real-world Analogy: Restaurant Service Time

Imagine measuring a restaurant's performance:

- Don't just take the average time a customer waits for food.
- A few long waits (like 45 mins) will skew the average

Instead:

- Measure how long 50% of customers waited (median = fair view)
- Also check how long the slowest 10% waited (95th percentile = worst-case view)

This tells you the real experience of most customers — like how users experience your website.

(7) Summary

- Mean is dragged by slow users.
- Median shows the typical user.
- 95th percentile shows worst-case experience.

⇒ This mirrors real website performance measurement.

② The slowdown Experiment Design

① What is a slowdown Experiment?

? Deliberately add a controlled amount of latency (e.g. +100ms, +200ms, +500ms) to the treatment group so you can measure how sensitive your key metrics are to page-speed changes. Amazon, Bing, and Google have all run public slowdown tests:

→ • Amazon saw ~1% sales for every extra 100ms of delay.

→ • Bing reported that a 2s slowdown cut revenue/user by ~4%

→ • Google showed 0.4–0.5s extra latency dropped traffic by ~20%.

② Core design steps (keep it simple → trustworthy)

Step	What you actually do	Why it matters.
1. Define OEC	Pick the Overall Evaluation Criterion you care about (e.g. conversion rate, ad revenue, search queries)	You'll judge the slowdown's damage with this single yard-stick.
2. Choose latency "doses"	Typical tiers: 0ms (control), +100ms, +250ms, +500ms	Lets you fit a dose-response curve instead of a single point.

3. Randomize by user	Each user is locked to one tier for the whole experiment	Avoids contamination and keeps treatments comparable.
4. Inject delay safely	Easiest: server-side sleep() before sending HTML, or client-side JS setTimeout before rendering.	Guarantees the only difference is latency — not HTML/css.
5. Run long enough	Use power calculations so every tier has (say) ≥ 250k users.	Small latency effects need big samples to rise above noise.
6. Monitor guard-rail metrics	Crash rate, errors logs, bounce rate. Abort if you break something	Protect users and your brand while "hurting" them on purpose.
7. Analyze slope	Fit a line (or spline) of metric vs. delay: the first-order slope = performance sensitivity.	This is the quantitative input for cost-of-latency decisions.

(3) Real-world analogy: training with ankle weights.

A track coach has runners do 0 kg (control), 1 kg, 2.5 kg, 2.5 kg and 5 kg ankle-weight laps.

- Same athletes, same track, only the weight changes
  → randomization & isolation.

- Coach measures lap times to trace a smooth curve: every extra kilogram slows the runner by $\approx \Delta$ seconds.

- High weight (1 kg) effects are almost linear; heavier weights (5 kg) show diminishing returns - mirroring local linear approximation vs "extreme results".

Local Linear Approximation vs. Extreme Results.

Concept	LLA	Extreme Results.
1. Definition	Assume small performance changes cause proportionally small metric changes.	At large delays, metrics can drop much faster and non-linearly.
2. Mathematical form	$M(t+\Delta t) \approx M(t) + M'(t) \cdot \Delta t$ - first order Taylor approximation	The simple linear model breaks - higher-order terms dominate.
3. Use case	Estimating the cost of a 50-200ms slowdowns	Understanding risks of large regressions (e.g. 1000 ms).
4. Assumption	The metric vs. delay curve is locally smooth and linear.	The curve becomes non linear, sometimes convex or flat.
5. Experimental Design	Use small treatment groups: 0ms, 100ms, 250ms, etc	Add an extreme tier: 1000 ms+, to expose behavior collapse.
6. Model.	Fit a straight line (linear regression)	Fit a nonlinear curve (e.g. quadratic spline)
7. Example	Conversion drops from 4.0% -> 3.95% when delay is +100ms	Conversion drops from 4.0% -> 3.0% when delay is +1000ms

8. Mental Model	Gentle slope = predictable performance cost	Tipping point = Catastrophic user drop-off.
9. Danger if misuse	Using LHA for large changes will underestimate damage	Using extreme results alone can scare you off small, safe changes.

4) The math that powers the design.

For a small added delay Δt, assumes a local linear drop in the metric M:

$$f(x+\Delta x) \approx f(x) + f'(x) \cdot \Delta x$$

becomes,

$$M(t+\Delta t) \approx M(t) + \underbrace{M'(t)}_{\text{sensitivity}} \cdot \Delta t$$

By running multiple latency tiers, you estimate $M'(t)$ (the slope) directly with simple linear regression.

$$\widehat{M't} = \frac{\Sigma_i (t_i - \bar{t})(M_i - \bar{M})}{\Sigma_i (t_i - \bar{t})^2}.$$

Where,

- $M(t)$: Metric (e.g., conversion rate) at delay t
- $M'(t)$: Slope - how much the metric changes per ms of delay.
- Δt : added delay in ms.

Feature	Derivative Process	Local Linear Approximation (LLA)	Extreme Results
What is it?	Exact slope from a known slope.	Approximate slope from real-world.	Behaviour under large delays where slope is no longer constant.
Used when	You know the exact function $M(t)$	You have experimental data over a small range (e.g. 0–250 ms)	You test very large delays (e.g. 1000 ms), possibly beyond LLA.
Mathematical tool	Calculus derivative $M'(t)$	Linear regression (least squares)	Non linear curve fitting (e.g. quadratic spline) or tie-wise comparison.
Assumes	Function is differentiable and noise-free.	Matrix is locally linear, data is noisy	Relationship is nonlinear (e.g. convex or exponential drop).
Works with	Symbolic math (e.g. simulations, models)	Real A/B testing data (0–250 ms)	Real A/B testing data with high delay (500–1000 ms)
Goal	Calculate the true instantaneous rate of change	Estimate the local effect per ms delay	Measure extreme behaviour, detect sharp breakdown.
Fails when	Function is unknown or data is noisy	Delay range is too large (curve bends)	used with small delays - it overreact
Best for	Theoretical modeling / prediction	Estimating slope in early latency zones	Diagnosing catastrophic latency effects

Formulas Comparison

Name	Key Formula	Examples.
Derivative	$M'(t) = \frac{dM}{dt}$	if $M(t) = 4 - 0.001t + 0.0000002t^2$, then $M'(t) = -0.001 + 0.0000004t$
LLA	$\hat{M}(t) = a + bt$, with b = slope from regression	Fit linear regression using data from 0, 100, 250 ms.
Extreme	$\hat{M}(t) = a + bt + ct^2$ or tier-by-tier comparison	Fit quadratic model or analyze large-delay tier separately (e.g. 100 ms).

Visual Analogy

Method	Analogy
Derivative	You have the map and slope equation of the hill — you can compute slope anywhere.
LLA	You walk 20 steps and estimate the slope from experience — you don't know the whole hill, just your local path.
Extreme Results	Suddenly the hill becomes a cliff — slope estimate from 20 steps back no longer applies, you need to switch models or stop hiking.

Math Examples:

1) Local Linear Approximation (LLA)

Step-by-Step Math

→ would use the three "small-delay" tians (0 ms, 100 ms, 250 ms).

tp delay (ms)	yi mean conversion %
0	4.00
100	3.94
250	3.85.

(1.) Compute means

$$\bar{t} = \frac{0+100+250}{3} = 116.7, \quad \bar{y} = \frac{4.00+3.94+3.85}{3} = 3.93$$

(2) Numerator & demominator for the slope

$$\sum (t_i - \bar{t})(y_i - \bar{y}) = (0-116.7)(4.00-3.93) + (100-116.7)(3.94-3.93) + (250-116.7)(3.85-3.93) \approx -34.25.$$

$$\sum (t_i - \bar{t})^2 = (0-116.7)^2 + (100-116.7)^2 + (250-116.7)^2 \approx 54.67$$

3. Slop & intercept:

$$b = -\frac{34.25}{34.167} = -0.001002$$

$$a = \bar{y} - b\bar{t} = 3.93 + 0.117 = 4.047$$

(4) LLA model

$$\hat{M}(t) = 4.047 - 0.001002\,t$$

Interpretation: Every extra 1 ms slows conversion ≈ 0.001 percentage-points (0.00001%)

Matching code (quick-run)

```
import numpy as np
t = np.array([0, 100, 250], dtype = float)
y = np.array([4.00, 3.94, 3.85])
b = np.cov(t, y, bias = True)[0, 1] / np.var(t)
a = y.mean() - b * t.mean()
print("LLA model: M(t) = %.3f %.6f t"
      % (a, b))
```

output: $M(t) = 4.000 - 0.000600\,t$.

2) Extreme Results

Step-by-Step Math

Now include the large-delay trials (500 ms, 1000 ms) and fit a quadratic, the simplest nonlinear that can bend:

t_i (ms)	y_i (%)
0	4.00
100	3.94
250	3.85
500	3.85
1000	1.45.

1. Set up quadratic model

$$y_i = a + b t_i + c t_i^2$$

2. Normal-equation matrices.

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 100 & 100^2 \\ 1 & 250 & 250^2 \\ 1 & 500 & 500^2 \\ 1 & 1000 & 1000^2 \end{bmatrix}, \quad y = \begin{bmatrix} 4.00 \\ 3.94 \\ 3.85 \\ 3.85 \\ 1.45 \end{bmatrix}$$

Solve $(X^T X)\beta = X^T y$ for $\beta = [a, b, c]^T$

3. Closed-form solution (number rounded)

$$a = 3.989, \; b = -0.00093, \; c = -1.51 \times 10^{-6}.$$

4. Quadratic

$$A(t) = 3.989 - 0.00093\, t - 1.51 \times 10^{-6}\, t^2$$

5. Why LLA fails at 1000 ms

- LLA prediction: $4.047 - 0.001002 \times 1000 \approx 3.05\%$.
- Quadratic (better) prediction $= 3.989 - 0.93 = 1.55\%$
- Actual mean (data): 1.45%.

The straight-line LLA under-estimates the damage by ≈ 1.6 percentage-points proving "extreme scrolls".

(E) Linked Concept 1:

Topic	Connection
Key Assumption: Local Linear Approx.	Slowdown time should be small enough that a st line is reasonable between adjacent points.
How to Measure Website Performance	You must log accurate per-user load times (e.g. median
Impact of Different Page Elements Differs.	You can run component-level slowdowns (e.g. only images vs only JS) to see which parts hurt most
Extreme Result	Very large delays (>1s) often cause super-linear drops: a separate "extreme" test can expose that non-linearity.

⑨ Impact of Different Page Elements Differs.

1. Concept in one breath.

Not every extra millisecond hurt equally. Delaying the first line of text (above-the-fold) frustrates users far more than delaying, say, a footer image.

Therefore the experiment inject latency into specific page components - HTML shell, CSS, hero image, ads, async JS - and measure each component's separate effect on business metrics.

2. Why the impact differ

Page elements (examples)	When user notices it	Typical cost of 100ms delay
Initial HTML / FCP	Immediately (blank screen)	-0.08% conversion
Hero image	Within 1s viewport	-0.05% conversion
Ads / Analytic JS	After page "looks" ready	-0.02% conversion
Below-the-fold images	Needs scroll	≈0 (often negligible)

** Numbers are illustrative; each site much measure it's own.

Key idea : user-perceived performance = what is delivered X when it becomes visible/interactive.

3) Mathematical framing

→ Treat the page as a bundle of k elements. Slow each element by Δt_j ms. Assume a locally linear response (just like LHA but with multiple slopes)

$$M = a + \sum_{j=1}^{k} b_j \Delta t_j + \varepsilon$$

- M = metric (e.g. conversion %)
- b_j = sensitivity of the metric to element j
- Larger $|b_j|$ ⇒ that element's delay hurts more.

This is just multiple linear regression; it lets you answer

> "If I optimise CSS delivery by 50ms, how much revenue do I gain, holding everything else constant?!"

(1) Detailed Explanations.

- Concept : Different page components hurt conversion differently

When a web page loads, it doesn't all render at once - it's composed of multiple parts like :

- HTML shell
- CSS stylesheets
- Hero image
- Ads
- Analytics script.

Each of these affects user-perceived performance differently depending on:

- When the user sees it (e.g. above vs. below the fold)
- What it does (e.g. blocks vs. lazy loads)
- Whether it blocks interactivity (main thread)

⇒ So, slowing HTML shell (first thing users see) has greater negative impact on conversion than delaying ads, even by the same 100 ms.

Thus, we model the total metric as the sum of each part's latency multiplied by its sensitivity.

(2) Mathematical Framing & Full Examples.

Let's suppose the metric M (e.g. conversion rate in %) is impacted by the slowdown of 3 elements

- Δt_1 : Delay added to HTML shell
- Δt_2 : Delay added to Hero image
- Δt_3 : Delay added to Ads.

We assume each has a linear effect.

$$M = a + b_1 \cdot \Delta t_1 + b_2 \cdot \Delta t_2 + b_3 \cdot \Delta t_3 + \epsilon$$

Where:

- a : baseline conversion rate (control group)
- b_1, b_2, b_3 : how much conversion rate drops per 1 ms of delay in each component.
- ϵ : random error (noise)

(a) Example Dataset

Test Group	Delay HTML (ms)	Delay Hero (ms)	Delay Ads (ms)	Conversion Rate (%)
A (control)	0	0	0	4.00
B	100	0	0	3.92
C	0	100	0	3.92
D	0	0	100	3.98

(b) Example Dataset.

we want to solve the linear system:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 100 & 0 & 0 \\ 1 & 0 & 100 & 0 \\ 1 & 0 & 0 & 100 \end{bmatrix} \cdot \begin{bmatrix} a \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 4.00 \\ 3.92 \\ 3.95 \\ 3.98 \end{bmatrix}$$

Break into individual equations.

$4.00 = a + 0 + 0 + 0 \rightarrow a = 4.00$

$3.92 = a + 100 \cdot b_1 \rightarrow b_1 = (3.92 - 4.00)/100 = -0.0008$

$3.95 = a + 100 \cdot b_2 \rightarrow b_2 = (3.92 - 4.00)/100 = -0.005$

$3.98 = a + 100 \cdot b_3 \rightarrow b_3 = (3.98 - 4.00)/100 = -0.0002$

(c) Final linear Model

$$M = 4.00 - 0.0008 \cdot \Delta t_{HTML} - 0.0005 \cdot \Delta t_{Hero} - 0.0002 \cdot \Delta t_{ads}$$

(d) Interpretation

Component	Slope bj	Meaning.
HTML	- 0.0008	Each 1ms delay in HTML, shell drops conversion by 0.0008 percentage points.
Hero	- 0.0005	Each 1ms delay in hero image drops conversion by 0.0005 pp
Ads	- 0.0002	Each 1ms delay in ads drop conversion by 0.0002 pp.

(e) Prediction Examples.

If we delay:

- HTML : 80ms
- Hero : 100 ms
- Ads : 150ms

Then expected conversion is:

$$M = 4.00 - 0.0008 \cdot 80 - 0.0005 \cdot 100 - 0.0002 \cdot 150$$

$$M = 4.00 - 0.064 - 0.05 - 0.03 = \boxed{3.856}$$

(4) Real-world Analogy.

Loading a news site is like serving a meal:

- Plate on table (HTML skeleton) - must arrive first! diners won't wait.
- Main dish (hero image / text) - still critical
- Condiments (ads/analytics) - nice but can appear later.
- Dessert (below-fold images) - many users leave before it matters.

Delaying each stage conveys dinners differently - so you measure each stage separately.

(6) How it interlocks with other Unit-5 topics

Topic	Overlap
LHA	You still assume each element's impact is approximately linear for small delays.
Slowdown Experiment Design	Requires component - selective latency injection (e.g. add 5 ms only to CSS route)
Extreme Results	Even low-value elements (ads) can cause huge drops if they block the main thread - non linear blow

blows ups appears here.

One-liner takeaways

Speed work is ~~here~~ triage: measure which page parts hurt most when slowed, then optimise those first.